

# Résumé du projet : PopJav

*Titre professionnel Développeur Web et Web Mobile — Enzo Gavini — La Plateforme, session 2026*

Introduction : PopJav est une plateforme web d'apprentissage du langage Java qui rend la montée en compétences progressive et ludique. Le contenu est organisé en chapitres et en leçons, chacune pouvant être validée par un quiz « à vies » inspiré des jeux vidéo : chaque erreur coûte une vie, un minuteur rythme les questions, et un seuil de réussite décide du résultat. L'application est construite en sept microservices Spring Boot derrière une passerelle qui valide les JWT et applique le contrôle d'accès par rôle, avec une persistance polyglotte PostgreSQL / MongoDB, le tout entièrement conteneurisé.

## ACTIVITÉ TYPE 1 : développer la partie front-end d'une application web ou web mobile sécurisée

### CP 1 — Installer et configurer son environnement de travail

- **Environnement** : IntelliJ IDEA avec le JDK Amazon Corretto 17 (LTS) ; projet découpé en sept modules Maven, chacun avec son pom.xml et le Maven Wrapper pour un build reproductible sur tout poste.
- **Versionnement** : Git avec dépôt GitHub, convention Conventional Commits (feat, fix, docs, test...) et branches thématiques fusionnées dans main.
- **Infrastructure locale** : PostgreSQL, MongoDB et Consul conteneurisés avec Docker à partir des images officielles, configurés par variables d'environnement centralisées dans un fichier .env.

### CP 2 — Maquetter des interfaces utilisateur

- **Démarche** : wireframes des écrans clés (catalogue, leçon, quiz, profil, administration) puis maquettes détaillées sur Figma, en versions mobile et desktop.
- **Mobile-first** : chaque écran est pensé d'abord pour mobile puis décliné en desktop ; les maquettes ont servi de référence pendant toute l'intégration.

### CP 3 — Réaliser des interfaces utilisateur statiques

- **Intégration** : l'ensemble des pages en HTML5 avec Thymeleaf et Tailwind CSS, complétés par des feuilles de style personnalisées versionnées (cache-busting ?v=).
- **Charte graphique centralisée** : variables CSS strictes (--violet, --rose, --or, rayons 14 px, boutons « pilule », ombres douces) ; typographies Baloo 2 (titres) et Poppins (interface et textes).
- **Factorisation** : en-tête, barre de navigation et pied de page en fragments Thymeleaf réutilisés par toutes les pages — une seule source de vérité.

- **Responsive** : Flexbox pour la structure générale et CSS Grid pour la grille de réponses du quiz : une colonne sur mobile, deux colonnes sur grand écran via une media query.
- **Accessibilité** : étiquettes de formulaires, contrastes, indicateur de focus clavier or de 3 px, lien d'évitement vers le contenu principal, attributs alternatifs sur les images.

#### CP 4 — Développer la partie dynamique des interfaces utilisateur

- **Rendu dynamique côté serveur** : pages générées à partir des données réelles (th:each), formulaires liés aux objets Java (th:object / th:field), affichage conditionnel selon le rôle en session — les actions d'administration ne sont visibles que pour les comptes ADMIN.
- **Quiz interactif** : un script pilote l'affichage question par question (Précédent / Suivant, compteur) avec un minuteur de 30 secondes par question qui soumet automatiquement le quiz à expiration.
- **Validation cliente accessible** : vérification des champs avant soumission (email, politique de mot de passe) avec zone d'erreur annoncée aux lecteurs d'écran (role="alert", aria-invalid) — en complément de la validation serveur, seule garantie de sécurité.
- **Navigation mobile** : menu burger accessible (aria-expanded tenu à jour, fermeture à la touche Échap) et confirmation avant toute suppression.

## ACTIVITÉ TYPE 2 : développer la partie back-end d'une application web ou web mobile sécurisée

#### CP 5 — Mettre en place une base de données relationnelle

- **Modélisation** : démarche MCD → MLD → MPD sous PostgreSQL 16 : utilisateurs, hiérarchie de contenu (chapitre → leçon → quiz → question → réponse) et résultats de quiz.
- **Intégrité portée par la base** : clés étrangères entre les tables de contenu et contraintes d'unicité sur l'email et le nom d'utilisateur — un contrôle applicatif contourné serait rattrapé par la base.
- **Déploiement** : conteneur Docker (image officielle), volume de persistance, healthcheck pg\_isready conditionnant le démarrage des services dépendants ; seul persistence-service y accède.

#### CP 6 — Développer des composants d'accès aux données SQL et NoSQL

- **SQL** : Spring Data JPA (Hibernate) : un repository par entité, méthodes dérivées (findByEmail, existsByUsername) traduites en requêtes préparées — protection contre l'injection SQL.
- **NoSQL** : commentaires gérés par Spring Data MongoDB : documents @Document et MongoRepository (findById, findByLessonId), un modèle documentaire adapté à des données autonomes au schéma souple.
- **Références logiques inter-contextes** : les identifiants croisés (Result.userId, Comment.userId/lessonId) sont volontairement sans clé étrangère physique : couplage

faible entre services et entre bases hétérogènes (pattern database-per-service), la cohérence étant garantie par la chaîne d'identité.

### CP 7 — Développer des composants métier côté serveur

- **Architecture en couches** : controllers → services → repositories dans chacun des sept microservices, qui se découvrent via Consul et communiquent en REST avec OpenFeign.
- **Moteur de quiz côté serveur** : le service récupère sa propre copie du quiz — on ne fait jamais confiance à un score calculé par le client — décompte les vies, arrête la partie à zéro vie, calcule le pourcentage (avec garde contre la division par zéro) et masque les bonnes réponses avant tout envoi.
- **Authentification** : politique de mot de passe validée côté serveur, hachage BCrypt (le clair ne quitte jamais auth-service), message générique anti-énumération, émission de JWT signés HS256.
- **Gestion d'erreurs** : exceptions métier typées converties en codes HTTP appropriés (409, 401, 400, 404).
- **Tests** : JUnit 5 et Mockito, dépendances simulées ; couverture JaCoCo de 88 à 89 % sur les modules métier ; code documenté en anglais (javadoc et commentaires de logique métier et de sécurité).

### CP 8 — Documenter le déploiement d'une application dynamique

- **Images** : chaque service est empaqueté avec Jib (plugin Maven, sans Dockerfile) et publié sur Docker Hub.
- **Orchestration** : un docker-compose unique pilote dix conteneurs (sept services + PostgreSQL + MongoDB + Consul) avec healthchecks et ordre de démarrage maîtrisé (depends\_on / service\_healthy).
- **Documentation** : README en anglais : prérequis, diagrammes d'architecture et de données, variables d'environnement (.env.example en modèle) et lancement en deux commandes (cp .env.example .env puis docker compose up -d).

## COMPÉTENCE TRANSVERSE : la sécurité de bout en bout

- **Chaîne d'identité** : à la connexion, auth-service émet un JWT signé (claims userId et rôle), stocké dans la session serveur du BFF — le navigateur ne voit jamais le token. Un intercepteur Feign le transmet à chaque appel ; la passerelle le valide localement (secret partagé) puis écrase systématiquement les en-têtes X-User-Id / X-User-Role avec les valeurs extraites du token : une identité forgée par un client est ignorée.
- **Contrôles d'accès** : RBAC au gateway (écritures de contenu et administration réservées au rôle ADMIN, route interne /credentials inaccessible aux utilisateurs) et contrôle d'appartenance anti-IDOR : un utilisateur ne lit que ses propres résultats, toute autre tentative reçoit un 403.

- **Validation en conditions réelles** : une grille de dix-sept tests d'API (Postman) couvre les cas passants et les refus attendus (401 sans token, 403 sur les ressources d'autrui ou les routes admin) — l'ensemble est conforme.